

Job Posting Management API

Role: Software Developer

Difficulty: junior

Project Type: fullstack

Duration: 3 days

Focus: api_development

Candidate: Rahul Sharma

Project Overview

Create a RESTful API to manage job postings with CRUD, authentication, and PDF export.

Technologies

- JavaScript
- TypeScript
- React
- Next.js
- Node.js
- Express.js
- Laravel (PHP)
- Tailwind CSS
- MongoDB
- JWT

Requirements

- Use existing stack
- Implement role based auth
- Export postings to PDF
- Document API with Swagger

Objectives

- Build secure CRUD API
- Integrate JWT auth
- Export to PDF
- Provide Swagger docs

Functional Requirements

- Create, read, update, delete postings
- Auth for admin and recruiter roles
- Generate PDF of posting
- Validate input
- Paginate listings
- Search by title

Non-Functional Requirements

- JWT token expiry 1h
- Rate limit 100 req/min
- Input validation with Joi
- Error handling middleware
- Unit tests for routes
- Responsive UI for admin panel

Modules

Auth

JWT login and role checks

- Login endpoint
- Middleware

Postings

CRUD operations

- Create, read, update, delete

PDF

Export postings

- Generate PDF

Docs

API documentation

- Swagger UI

Implementation Plan

Setup

- Initialize repo, install dependencies
- Configure TypeScript
- Set up Express

Auth

- Create login route
- Implement JWT middleware
- Define roles

Postings

- Define schema, CRUD routes
- Add validation
- Pagination

PDF & Docs

- Add PDF generation
- Add Swagger docs

Evaluation Criteria

API completeness (40%)

All CRUD endpoints functional

Security (30%)

JWT and role checks work

Code quality (20%)

Linting, comments, tests

Documentation (10%)

Swagger and README

Deliverables

- API source code
- Postman collection
- Swagger UI
- README
- Unit tests

Suggested Folder Structure

- src
- src/controllers
- src/models
- src/routes
- src/middleware
- src/utils
- tests
- public

API Endpoints

POST /auth/login

Login and get token

GET /postings

List postings

POST /postings

Create posting

GET /postings/:id

Get posting

PUT /postings/:id

Update posting

DELETE /postings/:id

Delete posting

GET /postings/:id/pdf

Export posting to PDF

GET /docs

Swagger UI

Database Design

users

Store user credentials

- email
- passwordHash
- role

postings

Job postings

- title
- description
- company
- location
- createdAt
- updatedAt

tokens

Refresh tokens

- userId
- token
- expiresAt

Testing Plan

- Unit tests for auth middleware
- Unit tests for CRUD routes
- Integration test for PDF export
- Test rate limiting

Candidate Fit

Rahul has the required JavaScript, TypeScript, Node.js, Express, JWT, and MongoDB skills, making this project a realistic fit for his junior level.